Министерство образования Республики Беларусь
Учреждение образования
«Брестский Государственный технический университет»
Кафедра ИИТ

Лабораторная работа №2
По дисциплине «ОМО»

Выполнил:
Студент 3-го курса
Группы АС-68
Жуков А.А.
Проверил:
Крощенко А.А.

Брест 2026

Цель работы: Изучить применение **линейной** и **логистической регрессии** для решения практических задач. Научиться обучать модели, оценивать их качество с помощью соответствующих метрик и интерпретировать результаты

Ход работы

Общее задание: выполнить задания по варианту (регрессия и классификация), построить все требуемые визуализации и рассчитать метрики, написать отчет, создать пул-реквест в репозиторий с кодом решения и отчетом в формате pdf.

Вариант 2

- **Регрессия (Прогнозирование медицинских расходов)**
    1. Medical Cost Personal Datasets
    2. Предсказать страховые выплаты (charges)
    3. **Задания:**
        - загрузите и обработайте категориальные признаки (например, sex, smoker);
        - обучите модель **линейной регрессии** для предсказания charges;
        - рассчитайте **MAE** (Mean Absolute Error) и **R^2**;
        - визуализируйте зависимость charges от bmi (индекс массы тела) с помощью диаграммы рассеяния и линии регрессии.
- **Классификация (Диагностика заболеваний сердца)**
    1. Heart Disease UCI
    2. Предсказать наличие у пациента болезни сердца (target)
    3. **Задания:**
        - загрузите данные и разделите их на обучающую и тестовую выборки;
        - обучите модель **логистической регрессии**;
        - оцените модель с помощью **Accuracy**, **Precision**, **Recall** и **F1-score**;
        - постройте **матрицу ошибок**.

Код программы для первого задания lab2.py:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

```
# 1. Загрузка и обработка категориальных признаков
df = pd.read_csv("medical_cost_personal_dataset.csv")
df["sex"] = df["sex"].map({"male": 1, "female": 0})
df["smoker"] = df["smoker"].map({"yes": 1, "no": 0})
df = pd.get_dummies(df, columns=["region"], drop_first=True,
dtype=int)

# 2. Разделение 80/20
test = df.sample(frac=0.2, random_state=42)
train = df.drop(test.index)

features = [c for c in df.columns if c != "charges"]
X_train = train[features].to_numpy(dtype=float)
X_test = test[features].to_numpy(dtype=float)
y_train = train["charges"].to_numpy(dtype=float)
y_test = test["charges"].to_numpy(dtype=float)

# 3. Линейная регрессия (добавляем столбец единиц для свободного
члена)
add_bias = lambda X: np.column_stack([np.ones(len(X)), X])
beta, *_ = np.linalg.lstsq(add_bias(X_train), y_train, rcond=None)
y_pred = add_bias(X_test) @ beta

# 4. Метрики
mae = np.mean(np.abs(y_test - y_pred))
r2 = 1 - np.sum((y_test - y_pred) ** 2) / np.sum((y_test -
y_test.mean()) ** 2)
print(f"MAE: {mae:.2f}")
print(f"R2 : {r2:.3f}")
print(pd.Series(beta[1:], index=features).sort_values())

# 5. Диаграмма рассеяния charges от bmi + линия регрессии
k, b = np.polyfit(df["bmi"], df["charges"], 1)
x_line = np.linspace(df["bmi"].min(), df["bmi"].max(), 100)

plt.figure(figsize=(8, 6))
plt.scatter(df["bmi"], df["charges"], c=df["smoker"], cmap="coolwarm",
alpha=0.6)
plt.plot(x_line, k * x_line + b, color="black", linewidth=2,
label="Линия регрессии")
plt.xlabel("BMI")
plt.ylabel("Charges")
plt.title("Зависимость charges от BMI (красные: курильщики)")
plt.legend()
plt.grid(alpha=0.3)
plt.show()
```

Код программы для второго задания lab2(2).py:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# 1. Загрузка, целевая переменная (0 = здоров, 1-4 = болезнь)
df = pd.read_csv("heart_disease_uci.csv")
y = (df["num"] > 0).astype(int).to_numpy()
X = df.drop(columns=["num", "id", "dataset"])
```

```
# 2. Категориальные признаки: пропуски -> "unknown", затем one-hot
cat_cols = X.select_dtypes(exclude="number").columns.tolist()  # включая fbs,
exang
num_cols = X.select_dtypes(include="number").columns.tolist()
X[cat_cols] = X[cat_cols].fillna("unknown").astype(str)
X = pd.get_dummies(X, columns=cat_cols, drop_first=True, dtype=float)

# 3. Разделение 80/20 (случайное)
rng = np.random.default_rng(42)
idx = rng.permutation(len(X))
n_test = int(0.2 * len(X))
test_idx, train_idx = idx[:n_test], idx[n_test:]
X_train, X_test = X.iloc[train_idx].copy(), X.iloc[test_idx].copy()
y_train, y_test = y[train_idx], y[test_idx]

# 4. Пропуски в числовых: медиана по train; масштабирование по train
med = X_train[num_cols].median()
X_train[num_cols] = X_train[num_cols].fillna(med)
X_test[num_cols] = X_test[num_cols].fillna(med)
mean, std = X_train[num_cols].mean(), X_train[num_cols].std()
X_train[num_cols] = (X_train[num_cols] - mean) / std
X_test[num_cols] = (X_test[num_cols] - mean) / std

add_bias = lambda A: np.column_stack([np.ones(len(A)),
A.to_numpy(dtype=float)])
Xb_train, Xb_test = add_bias(X_train), add_bias(X_test)

# 5. Логистическая регрессия (градиентный спуск, L2-регуляризация)
sigmoid = lambda z: 1 / (1 + np.exp(-z))
w = np.zeros(Xb_train.shape[1])
lr, lam = 0.1, 0.01
for _ in range(5000):
    p = sigmoid(Xb_train @ w)
    grad = Xb_train.T @ (p - y_train) / len(y_train)
    reg = lam * w
    reg[0] = 0                        # свободный член не регуляризуем
    w -= lr * (grad + reg)

y_pred = (sigmoid(Xb_test @ w) >= 0.5).astype(int)

# 6. Матрица ошибок и метрики
tp = np.sum((y_pred == 1) & (y_test == 1))
tn = np.sum((y_pred == 0) & (y_test == 0))
fp = np.sum((y_pred == 1) & (y_test == 0))
fn = np.sum((y_pred == 0) & (y_test == 1))

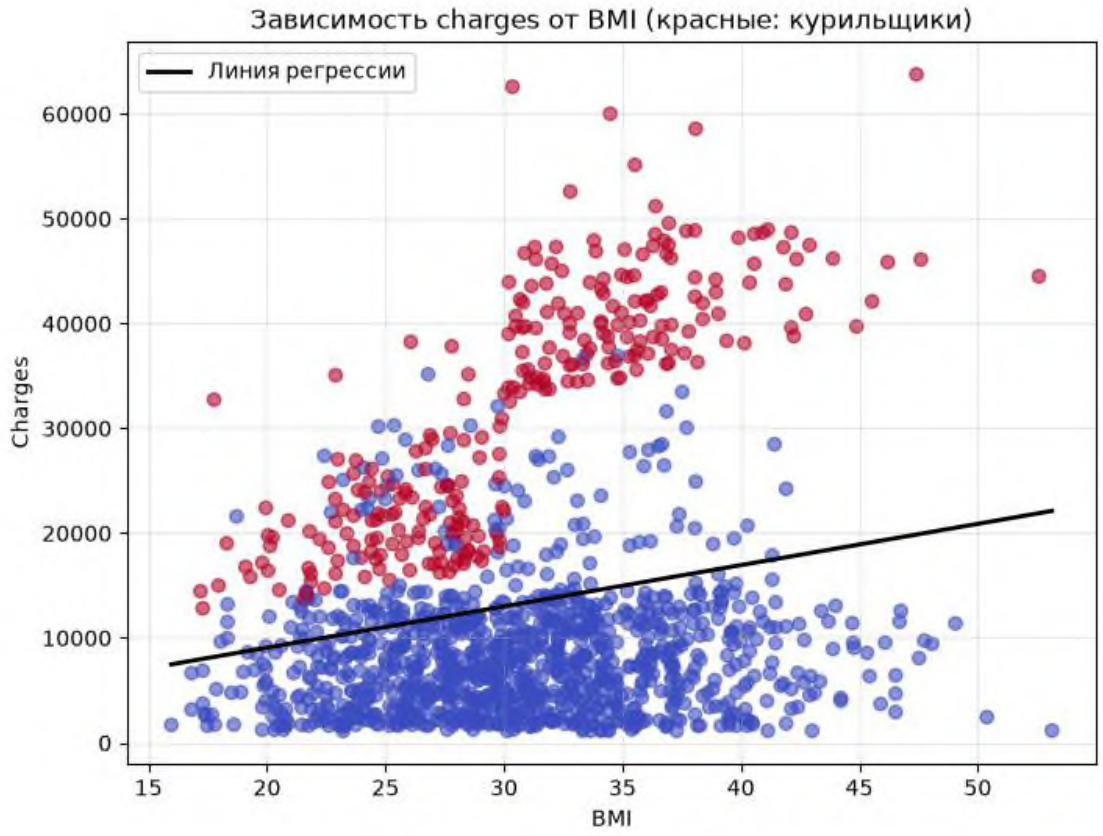
accuracy = (tp + tn) / len(y_test)
precision = tp / (tp + fp)
recall = tp / (tp + fn)
f1 = 2 * precision * recall / (precision + recall)

print(f"Accuracy : {accuracy:.3f}")
print(f"Precision: {precision:.3f}")
print(f"Recall   : {recall:.3f}")
print(f"F1-score : {f1:.3f}")

cm = np.array([[tn, fp], [fn, tp]])
fig, ax = plt.subplots(figsize=(5, 4))
ax.imshow(cm, cmap="Blues")
ax.set_xticks([0, 1], ["Здоров", "Болен"])
ax.set_yticks([0, 1], ["Здоров", "Болен"])
ax.set_xlabel("Предсказано")
ax.set_ylabel("Истинное значение")
```

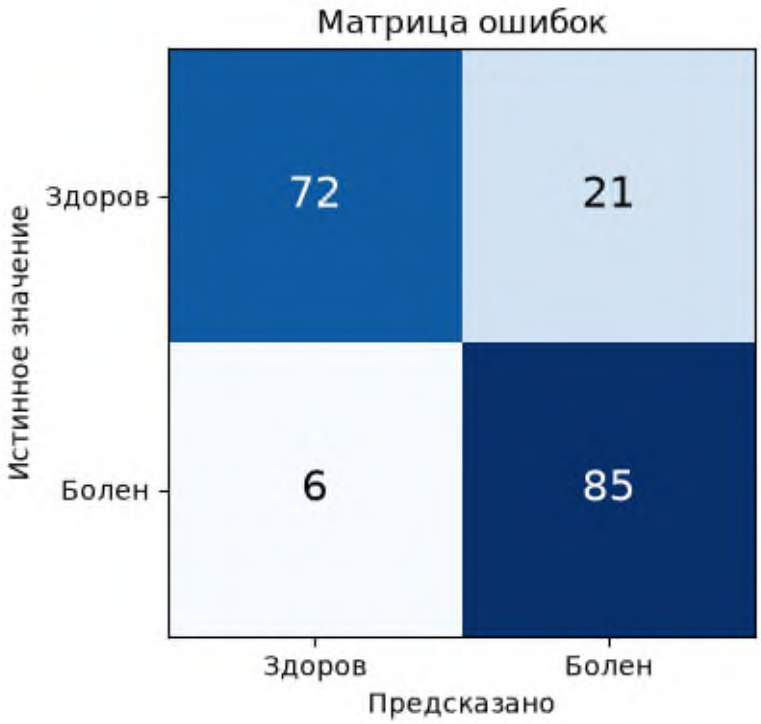
```
ax.set_title("Матрица ошибок")
for i in range(2):
    for j in range(2):
        ax.text(j, i, cm[i, j], ha="center", va="center", fontsize=16,
                color="white" if cm[i, j] > cm.max() / 2 else "black")
plt.tight_layout()
plt.show()
```

Результат первого задания:



MAE: 4181.19
R2 : 0.784
region_southwest -809.799354
region_southeast -657.864297
region_northwest -370.677326
sex -18.591692
age 256.975706
bmi 337.092552
children 425.278784
smoker 23651.128856

Результат второго задания



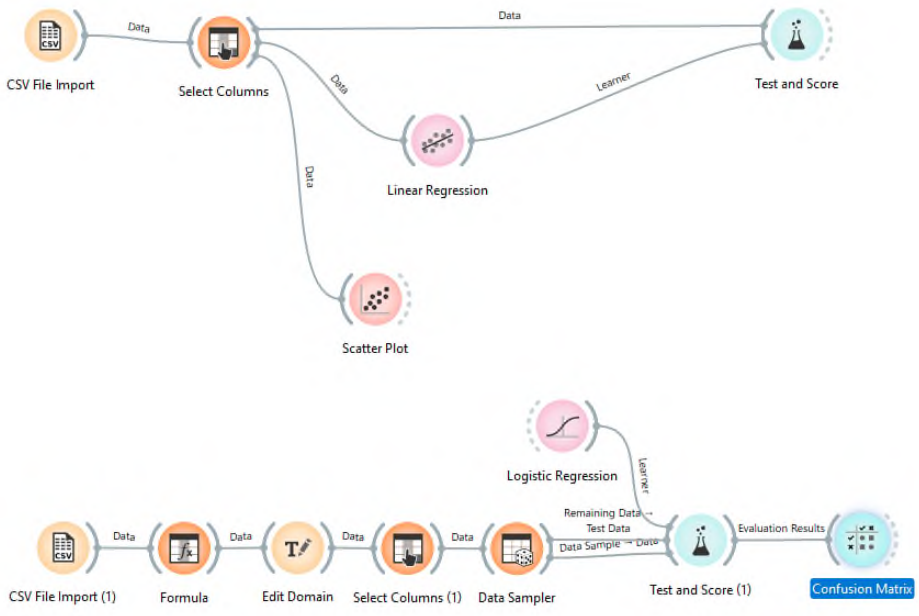
Accuracy : 0.853

Precision: 0.802

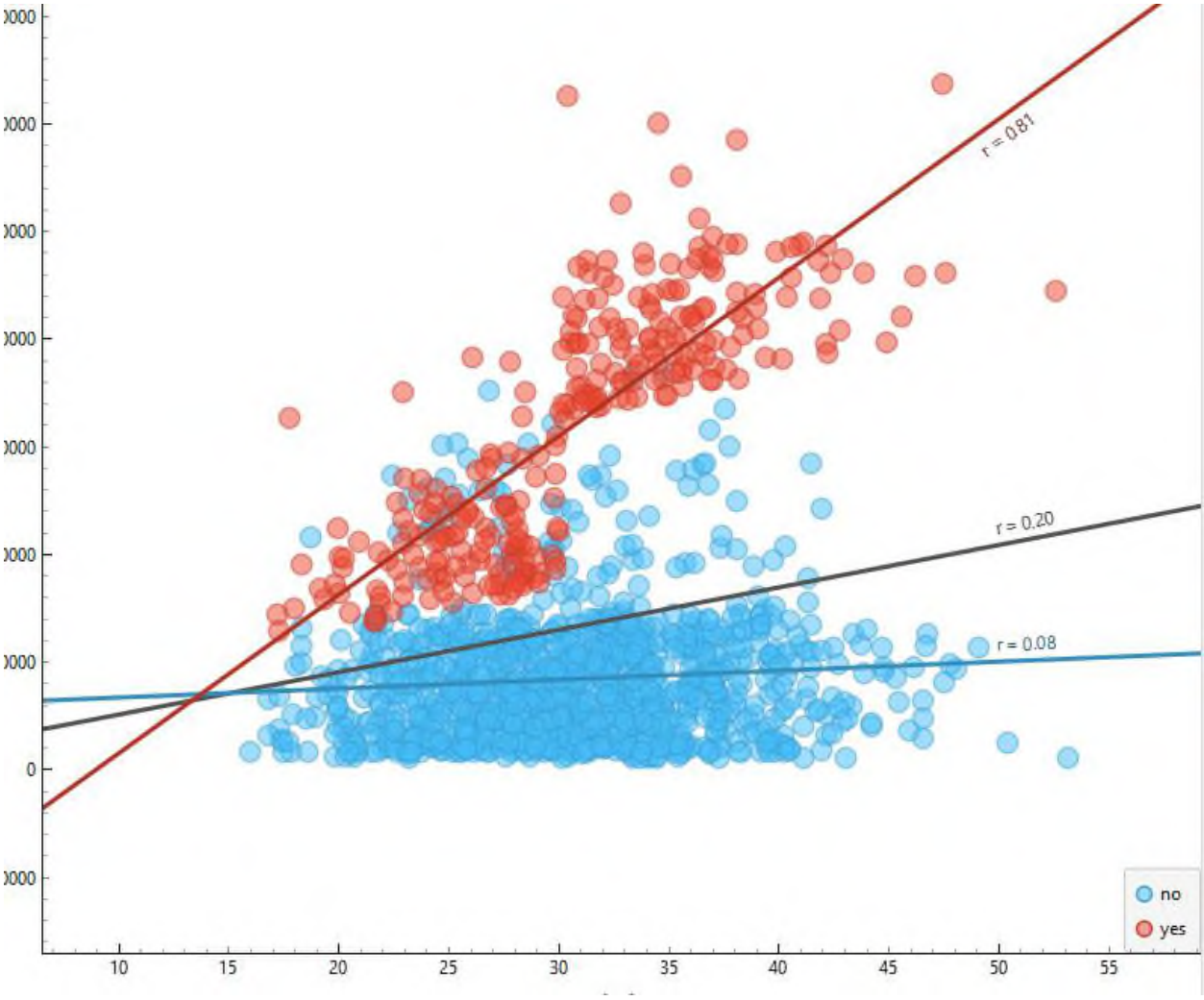
Recall : 0.934

F1-score : 0.863

Результаты в Orange:



Actual \ Predicted	0.0	1.0	Σ
0.0	65	17	82
1.0	19	83	102
Σ	84	100	184



Вывод: в ходе выполнения лабораторной работы были успешно освоены методы линейной и логистической регрессии для решения практических задач. Были обучены модели для задач регрессии и классификации, рассчитаны ключевые метрики их качества и проанализированы полученные результаты.